

Developing Resume Reviewer using LLMs

Overview



Mariana Anderson

Marketing Manager

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam pharetra in lorem at laoreet. Donec hendrerit libero eget est tempor, quis tempus arcu elementum. In elementum elit at dui tristique feugiat. Mauris convallis, mi at mattis malesuada, neque nulla volutpat dolor, hendrerit faucibus eros nibh ut nunc.

Contact

Phone
123-456-7890

Email
hello@reallygreatsite.com

Address
123 Anywhere St., Any City

Education

2008
Enter Your Degree
University/College

2008
Enter Your Degree
University/College

Expertise

- UI/UX
- Visual Design
- Wireframes
- Storyboards
- User Flows
- Process Flows

Language

English
Spanish

Experience

2019 - 2022
Company Name | 123 Anywhere St., Any City
Job position here

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam pharetra in lorem at laoreet. Donec hendrerit libero eget est tempor, quis tempus arcu elementum. In elementum elit at dui tristique feugiat. Mauris convallis, mi at mattis malesuada, neque nulla volutpat dolor, hendrerit faucibus eros nibh ut nunc. Proin luctus urna id nunc sagittis dignissim. Sed in libero sed libero dictum dapibus. Vivamus fermentum est eget lorem aliquet, vel tempus metus dignissim. Donec risus arcu, tristique et sollicitudin blandit, laculis ut nisi. Integer rutrum ultricies fringilla.

2017 - 2019
Company Name | 123 Anywhere St., Any City
Job position here

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam pharetra in lorem at laoreet. Donec hendrerit libero eget est tempor, quis tempus arcu elementum. In elementum elit at dui tristique feugiat. Mauris convallis, mi at mattis malesuada, neque nulla volutpat dolor, hendrerit faucibus eros nibh ut nunc. Proin luctus urna id nunc sagittis dignissim. Sed in libero sed libero dictum dapibus. Vivamus fermentum est eget lorem aliquet, vel tempus metus dignissim. Donec risus arcu, tristique et sollicitudin blandit, laculis ut nisi. Integer rutrum ultricies fringilla.

2015 - 2017
Company Name | 123 Anywhere St., Any City
Job position here

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam pharetra in lorem at laoreet. Donec hendrerit libero eget est tempor, quis tempus arcu elementum. In elementum elit at dui tristique feugiat. Mauris convallis, mi at mattis malesuada, neque nulla volutpat dolor, hendrerit faucibus eros nibh ut nunc. Proin luctus urna id nunc sagittis dignissim. Sed in libero sed libero dictum dapibus. Vivamus fermentum est eget lorem aliquet, vel tempus metus dignissim. Donec risus arcu, tristique et sollicitudin blandit, laculis ut nisi. Integer rutrum ultricies fringilla.

Reference

Name Surname
Job position, Company Name
Phone: 123-456-7890
Email: hello@reallygreatsite.com

Name Surname
Job position, Company Name
Phone: 123-456-7890
Email: hello@reallygreatsite.com

Job Description

Sales and Marketing Manager

Post & Job Title

- Sales and Marketing Manager

Location

- BioCity, Nottingham

Working Hours

- Flexible – 20 hours per week normally 4 hours per day

Reporting To

- Managing Director

Responsible For

- Sales and Marketing Assistant

Working With

- Managing Director and all other employees

Purpose of Job

- To manage the receipt, handling, and escalation of all sales enquiries
- To administer the preparation of quotes
- To create and implement a Marketing and PR strategy
- To identify and co-ordinate market information on behalf of HEC
- To produce appropriate marketing collateral as required
- To produce appropriate PR collateral as required
- Manage the day to day work of the Sales and Marketing Assistant
- Other tasks as required

Main Duties and Responsibilities

- To manage the receipt, handling, and escalation of all sales enquiries
 - To receive and co-ordinate all sales enquiries in line with the company's sales process
 - To monitor the status of all live enquiries to ensure all HEC and enquirer actions are completed in a timely professional manner and escalated to the MD when required
 - To ensure that all quotes and associated files are linked to an enquiry
 - To follow up sales activities with clients & potential clients
 - To chase and analyse the reasons for failure to turn an enquiry into a new order
 - To monitor company sales KPIs and sales performance

Overview

- Scoring how good a resume is against a set of JDs
- Suggestions on how to improve resume formatting
- Suggestions on how to improve gaps in various skill sets
- Recommendations on what is missing

Why use LLMs for analysing resumes?

- Efficient Resume Screening
- Enhanced Accuracy
- Bias Reduction
- Customizable Insights
- Scalable Processing
- Comprehensive Resume Analysis

What are we building?

- PDF/Image parser
- Text representation -> Embeddings
- LLM reviewer
- Chat application
 - Support for file upload & read

Langchain

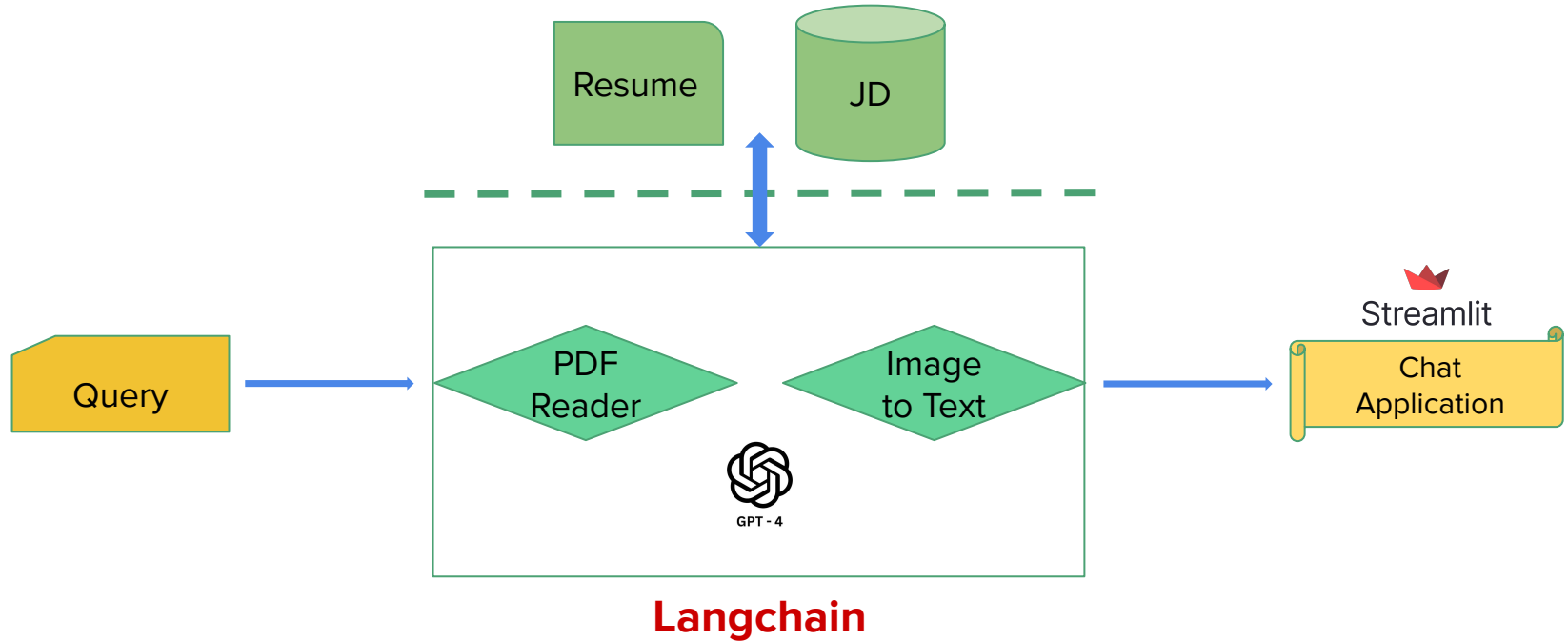
LangChain is an open source orchestration framework for the development of applications using large language models (LLMs).

Available in both Python- and Javascript-based libraries, LangChain's tools and APIs simplify the process of building LLM-driven applications like chatbots and virtual agents.



LangChain

Solution Approach



Tools and Tech

- **CSV/PDF** - Raw Data
- **OCR** - Image to Text
- **Python** - Coding language
- **GPT-4** - LLM
- **Langchain** - Orchestration framework
- **Streamlit** - Chat Application

System Setup

A Python Virtual Environment is an isolated space where you can work on your Python projects, separately from your system-installed Python.

Why do we need a virtual environment?

Imagine you are working on two different Python projects one of them uses Pandas 2.0 and the other uses Pandas 1.1

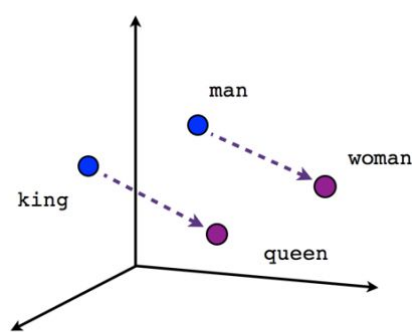
In such situations, we need to create a virtual environment in Python that can be really useful to maintain the dependencies of both projects.

How to setup the environment?

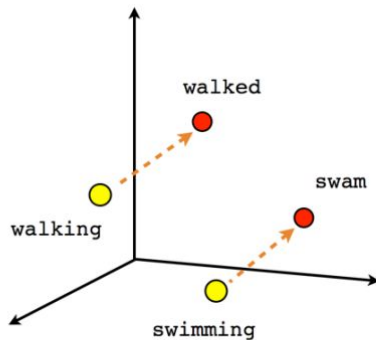
- `$ pip install virtualenv`
- `$ virtualenv my_env`
- activate virtual environment (Pycharm / VS Code)

What are Embeddings?

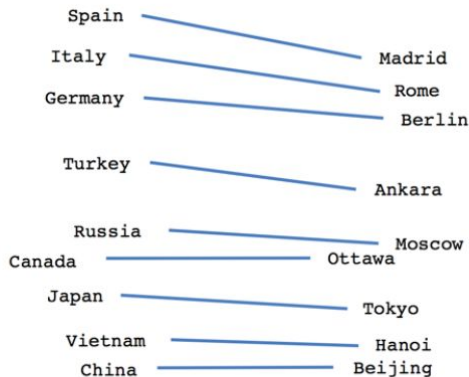
Embeddings are a way to represent items (like words, images, or other data) as vectors in a high-dimensional space, where similar items are closer together, and dissimilar items are farther apart.



Male-Female



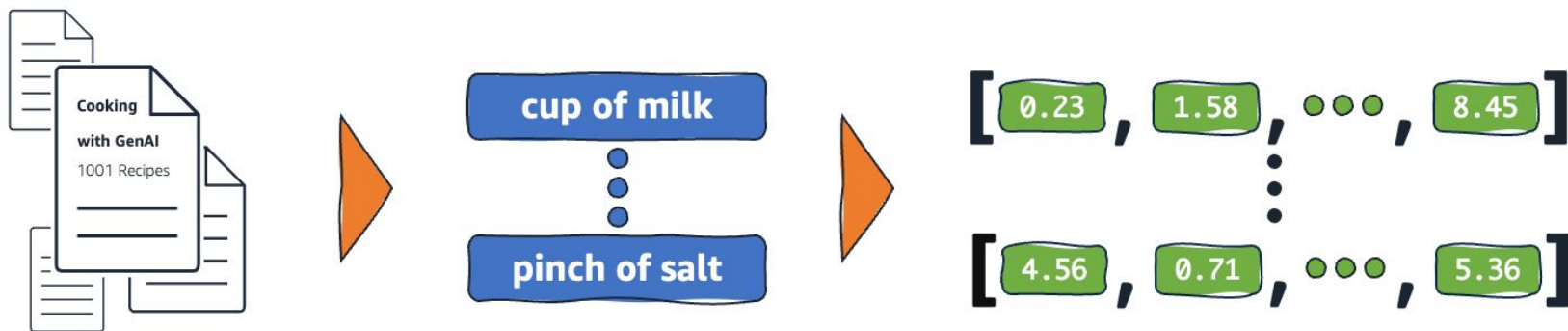
Verb tense



Country-Capital

What are Embeddings?

Embeddings convert raw data into continuous values that ML models can interpret.



Applications

Embeddings are commonly used for:

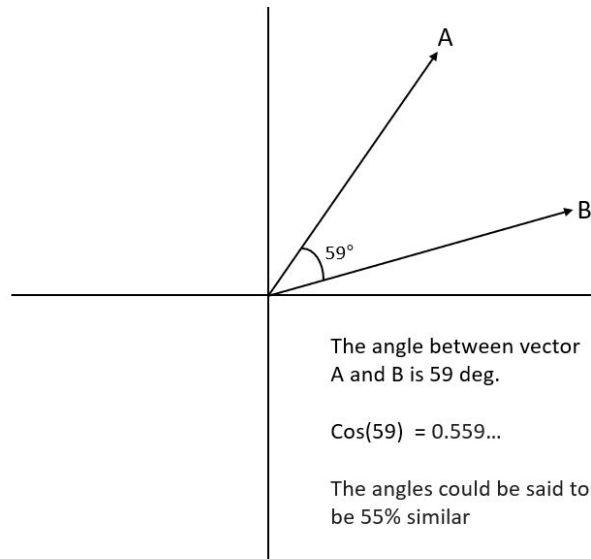
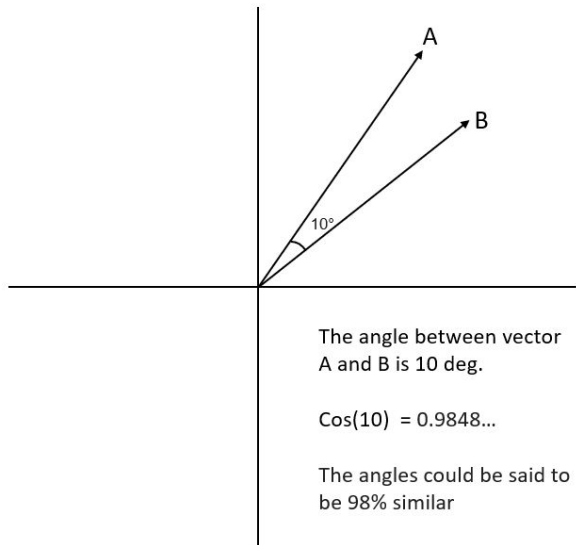
- **Search** (where results are ranked by relevance to a query string)
- **Clustering** (where text strings are grouped by similarity)
- **Recommendations** (where items with related text strings are recommended)
- **Anomaly** detection (where outliers with little relatedness are identified)
- **Classification** (where text strings are classified by their most similar label)

text-embedding-3-large by **OpenAI**

Model	Dimension	Max token	Knowledge cutoff	Pricing (\$/1k tokens)	MIRACL average	MTEB average
ada v2				0.0001	31.4	61.0
text-embedding-3-small	1536	8191	September 2021	0.00002	44.0	62.3
text-embedding-3-large	3072			0.00013	54.9	64.6

Cosine Similarity

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



Prompt Engineering

Prompt engineering is the process of designing, refining, and optimizing the inputs (prompts) given to a language model (like GPT) to achieve desired outputs.

Since language models generate responses based on the text they're given, crafting effective prompts is crucial for getting the most accurate, relevant, and useful results.

Prompt Engineering - Tactics

- Include details in your query to get more relevant answers
- Ask the model to adopt a persona
- Use delimiters to clearly indicate distinct parts of the input
- Specify the steps required to complete a task
- Provide examples
- Specify the desired length of the output

Prompt Engineering - Techniques

1. Zero-Shot Prompting
2. Few-Shot Prompting
3. Chain-of-Thought Prompting

Langchain

LangChain is a framework designed to help developers build applications that leverage large language models (LLMs) more effectively.

It provides tools and abstractions that simplify the process of integrating LLMs into complex workflows, such as chatbots, data analysis tools, or AI-driven decision-making systems.



Ref - <https://python.langchain.com/v0.2/docs/introduction/>

Installation and Setup

```
$ pip install langchain
```

Langchain Components

- **Model I/O:** Manages efficient interactions with various language models, optimizing input and output handling.
- **Retrieval:** Provides access to application-specific data, enabling dynamic data utilization.
- **Agents:** Directs applications in selecting appropriate tools based on high-level directives, enhancing decision-making.
- **Chains:** Supplies reusable, pre-defined compositions as foundational blocks for application development.
- **Memory:** Preserves application state across chain executions, ensuring context-aware interactions.

Model I/O

Prompts: Formatting for LLM inputs that guide generation

Chat models: Interfaces for language models that use chat messages as inputs and returns chat messages as outputs (as opposed to using plain text).

LLMs: Interfaces for language models that use plain text as input and output

Chat Models

The chat model interface is based around messages rather than raw text. The types of messages currently supported in LangChain are *AIMessage*, *HumanMessage*, *SystemMessage*, *FunctionMessage* and *ChatMessage*.

Most of the time, you'll just be dealing with

- **HumanMessage**
- **AIMessage**
- **SystemMessage**

```
from langchain.schema.messages import HumanMessage, SystemMessage
messages = [
    SystemMessage(content="You are Micheal Jordan."),
    HumanMessage(content="Which shoe manufacturer are you associated with?"),
]
response = chat.invoke(messages)
print(response.content)
```

I am associated with the Nike brand.

Output Parsers

Output parsers play a crucial role in Langchain, enabling users to structure the responses generated by language models.

- *PydanticOutputParser*
- *SimpleJsonOutputParser*
- *CommaSeparatedListOutputParser*

PydanticOutputParser

```
# Define your desired data structure using Pydantic
class Joke(BaseModel):
    setup: str = Field(description="question to set up a joke")
    punchline: str = Field(description="answer to resolve the joke")

    @validator("setup")
    def question_ends_with_question_mark(cls, field):
        if field[-1] != "?":
            raise ValueError("Badly formed question!")
        return field

# Set up a PydanticOutputParser
parser = PydanticOutputParser(pydantic_object=Joke)

# Create a prompt with format instructions
prompt = PromptTemplate(
    template="Answer the user query.\n{format_instructions}\n{query}\n",
    input_variables=["query"],
    partial_variables={"format_instructions": parser.get_format_instructions()},
)

# Define a query to prompt the language model
query = "Tell me a joke."

# Combine prompt, model, and parser to get structured output
prompt_and_model = prompt | model
output = prompt_and_model.invoke({"query": query})

# Parse the output using the parser
parsed_result = parser.invoke(output)

# The result is a structured object
print(parsed_result)

setup='Why did the chicken cross the road?' punchline='To get to the other side!'
```

SimpleJsonOutputParser

```
: from langchain.output_parsers.json import SimpleJsonOutputParser

# Create a JSON prompt
json_prompt = PromptTemplate.from_template(
    "Return a JSON object with `birthdate` and `birthplace` key that answers the following question: {question}"
)

# Initialize the JSON parser
json_parser = SimpleJsonOutputParser()

# Create a chain with the prompt, model, and parser
json_chain = json_prompt | model | json_parser

# Stream through the results
result_list = list(json_chain.stream({"question": "When and where was Elon Musk born?"}))

# The result is a list of JSON-like dictionaries
print(result_list)

[{'birthdate': 'June 28, 1971', 'birthplace': 'Pretoria, South Africa'}]
```

CommaSeparatedListOutputParser

```
from langchain.output_parsers import CommaSeparatedListOutputParser
from langchain.prompts import PromptTemplate
from langchain.llms import OpenAI

# Initialize the parser
output_parser = CommaSeparatedListOutputParser()

# Create format instructions
format_instructions = output_parser.get_format_instructions()

# Create a prompt to request a list
prompt = PromptTemplate(
    template="List five {subject}.\n{format_instructions}",
    input_variables=["subject"],
    partial_variables={"format_instructions": format_instructions}
)

# Define a query to prompt the model
query = "English Premier League Teams"

# Generate the output
output = model(prompt.format(subject=query))

# Parse the output using the parser
parsed_result = output_parser.parse(output)

# The result is a list of items
print(parsed_result)
```

```
['Manchester United', 'Liverpool', 'Chelsea', 'Arsenal', 'Manchester City']
```

Retrieval

Document Loaders

- Text File Loader
- CSV Loader
- PDF Loader
- Integrated Loaders - Slack, Notion, Google Drive, etc

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("example_data/layout-parser-paper.pdf")
pages = loader.load_and_split()
```

Document Transformers

Document transformers in LangChain are essential tools designed to manipulate documents

One such tool is the *RecursiveCharacterTextSplitter*, a versatile text splitter that uses a character list for splitting. It allows parameters like chunk size, overlap, and starting index.

```
text_splitter = RecursiveCharacterTextSplitter(  
    # Set a really small chunk size, just to show.  
    chunk_size=100,  
    chunk_overlap=20,  
    length_function=len,  
    is_separator_regex=False,  
)
```


Retrievers

A retriever is an interface that returns documents given an unstructured query. It is more general than a vector store.

A retriever does not need to be able to store documents, only to return (or retrieve) them.

Vector stores can be used as the backbone of a retriever, but there are other types of retrievers as well.

- *Chroma* retriever
- *ContextualCompressionRetriever* - Contextual Compression in LangChain compresses retrieved documents using the context of the query, ensuring only relevant information is returned. This involves content reduction and filtering out less relevant documents.

Agents

Agents leverage language models to dynamically determine sequences of actions to perform, making them incredibly versatile and adaptive.

The inputs to an agent typically include:

- **Tools:** Tools are interfaces that an agent can use to interact with the world.
- **User Input:** The high-level objective or query from the user.
- **Intermediate Steps:** A history of (action, tool output) pairs executed to reach the current user input.

Tools

```
: from langchain.tools import DuckDuckGoSearchRun
search = DuckDuckGoSearchRun()
search.run("manchester united vs luton town match summary")
```



```
: "Isaac Stacey Stronge. Saturday 11 November 2023 09:18. Manchester United emerged victorious when Luton Town visited Old Trafford for the first time in Premier League history. A single goal from ... Live coverage of the Manchester United vs. Luton Town English Premier League game on ESPN, including live score, highlights and updated stats. Manchester United 1-0 Luton Town (Nov 11, 2023) Game Analysis - ESPN Full Scoreboard » ESPN Expert recap and game analysis of the Luton Town vs. Manchester United English Premier... Manchester United Luton Town Latest Premier League Odds Stats Table Betting More Form Man Utd 3 - 4 vs FC Cop'hagen (a) 8 Nov 2023 UEFA Champions League Group A 1 - 0 vs Fulham (a) 4... Premier League match Man Utd vs Luton 11.11.2023. Preview and stats followed by live commentary, video highlights and match report."
```

```
: from langchain.tools import ShellTool
shell_tool = ShellTool()
print(shell_tool.run({"commands": ["echo 'Hello World!'", "time"]}))
```

Hello World!

```
real    0m0.001s
user    0m0.000s
sys     0m0.000s
```

Agents

The AgentExecutor is the runtime environment for an agent.

It is responsible for calling the agent, executing the actions it selects, passing the action outputs back to the agent, and repeating the process until the agent finishes.

Agents

```
from langchain.agents import tool

@tool
def get_word_length(word: str) -> int:
    """Returns the length of a word."""
    return len(word)

get_word_length.invoke("abc")
```

```
llm_with_tools = llm.bind_tools(tools)

from langchain.agents.format_scratchpad.openai_tools import (
    format_to_openai_tool_messages,
)
from langchain.agents.output_parsers.openai_tools import OpenAIToolsAgentOutputParser

agent = (
    {
        "input": lambda x: x["input"],
        "agent_scratchpad": lambda x: format_to_openai_tool_messages(
            x["intermediate_steps"]
        ),
    }
    | prompt
    | llm_with_tools
    | OpenAIToolsAgentOutputParser()
)
```

Prebuilt Agents

- Gmail
- SQL Database Agent
- Pandas DataFrame Agent

```
import pandas as pd
from langchain_openai import OpenAI

df = pd.read_csv(
    "https://raw.githubusercontent.com/pandas-dev/pandas/main/doc/data/titanic.csv"
)

agent = create_pandas_dataframe_agent(
    ChatOpenAI(temperature=0, model="gpt-3.5-turbo-0613"),
    df,
    verbose=True,
    agent_type=AgentType.OPENAI_FUNCTIONS,
)

agent.invoke("how many rows are there?")
```

Chains

Chains refer to sequences of calls - whether to an LLM, a tool, or a data preprocessing step. The primary supported way to do this is with LCEL.

```
from langchain.prompts import ChatPromptTemplate
from langchain.schema import StrOutputParser

model = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
prompt = ChatPromptTemplate.from_messages([
    ("system", "You're a very knowledgeable historian who provides accurate and eloquent answers to historical questions."),
    ("human", "{question}")
])
runnable = prompt | model | StrOutputParser()

for chunk in runnable.stream({"question": "What are the seven wonders of the world"}):
    print(chunk, end="", flush=True)
```

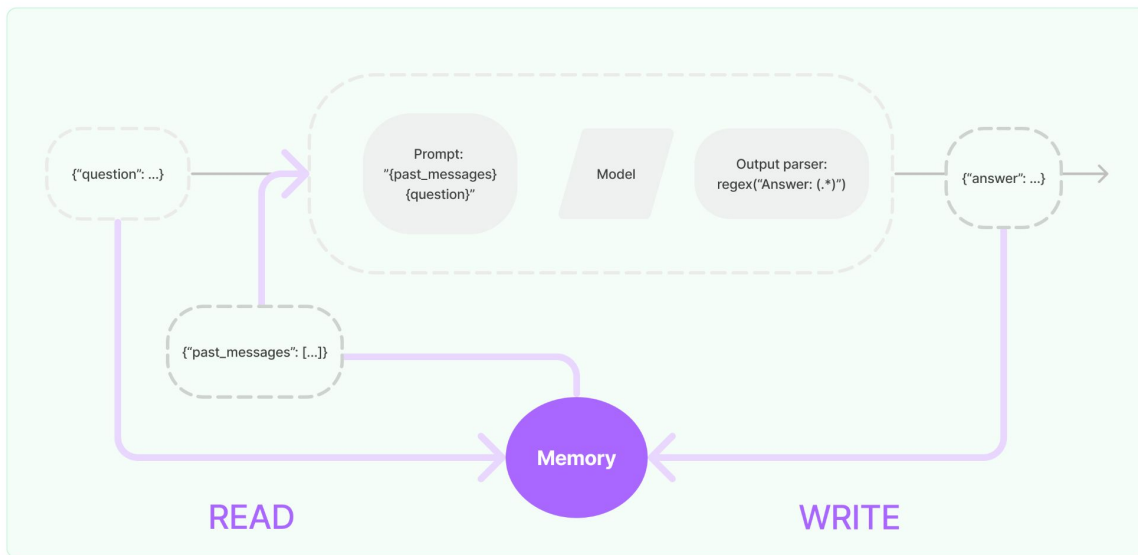
The Seven Wonders of the Ancient World were a collection of remarkable structures and monuments that were considered marvels of human ingenuity and craftsmanship in the ancient world. Unfortunately, only one of these wonders, the Great Pyramid of Giza, still stands today. The original list, compiled by various ancient Greek writers, included the following:

1. The Great Pyramid of Giza: The oldest and largest of the three pyramids in Giza, Egypt, built as a tomb for the Pharaoh Khufu around 2560 BCE.
2. The Hanging Gardens of Babylon: A terraced garden complex believed to have been built in the ancient city of Babylon, present-day Iraq, by King Nebuchadnezzar II in the 6th century BCE.
3. The Statue of Zeus at Olympia: A colossal seated statue of the Greek god Zeus, created by the renowned sculptor Phidias and placed in the Temple of Zeus in Olympia, Greece, around 435 BCE.
4. The Temple of Artemis at Ephesus: A grand temple dedicated to the Greek goddess Artemis, located in Ephesus, present-day Turkey. It was destroyed and rebuilt multiple times, with the final version completed around 550 BCE.
5. The Mausoleum at Halicarnassus: A monumental tomb built for Mausolus, the ruler of Caria in present-day Turkey, and his wife Artemisia. It

Memory

In LangChain, memory is a fundamental aspect of conversational interfaces, allowing systems to reference past interactions.

This is achieved through storing and querying information, with two primary actions: reading and writing.



ConversationBufferMemory

```
from langchain_openai import OpenAI
from langchain_core.prompts import PromptTemplate
from langchain.chains import LLMChain
from langchain.memory import ConversationBufferMemory

llm = OpenAI(temperature=0)
# Notice that "chat_history" is present in the prompt template
template = """You are a nice chatbot having a conversation with a human.

Previous conversation:
{chat_history}

New human question: {question}
Response: """
prompt = PromptTemplate.from_template(template)
# Notice that we need to align the `memory_key`
memory = ConversationBufferMemory(memory_key="chat_history")
conversation = LLMChain(
    llm=llm,
    prompt=prompt,
    verbose=True,
    memory=memory
)
```

Other Memory Types

- **Conversation Buffer Window Memory**
 - keeps a list of recent interactions and uses the last K interactions, preventing the buffer from getting too large
- **Conversation Entity Memory**
 - remembers facts about specific entities in a conversation and extracts information using an LLM.
- **Conversation Knowledge Graph Memory**
 - uses a knowledge graph to recreate memory. You can extract current entities and knowledge triplets from messages
- **Conversation Summary Memory**
 - creates a summary of the conversation over time, useful for condensing information from longer conversations

Streamlit

Streamlit is an open-source Python library that allows you to create and share custom web apps for machine learning and data science projects with minimal code.

It's particularly popular among data scientists and developers because it simplifies the process of turning data scripts into interactive web applications.

Installation and Setup

```
$ pip install streamlit
```

Demo